

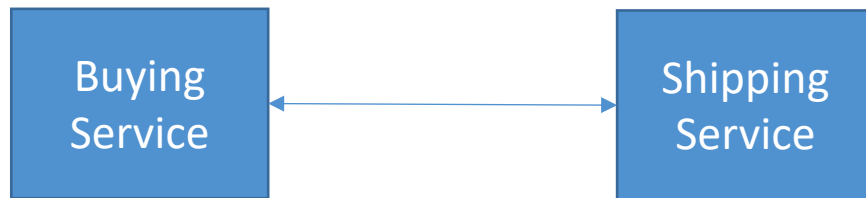
AWS Integration & Messaging

SQS, SNS & Kinesis

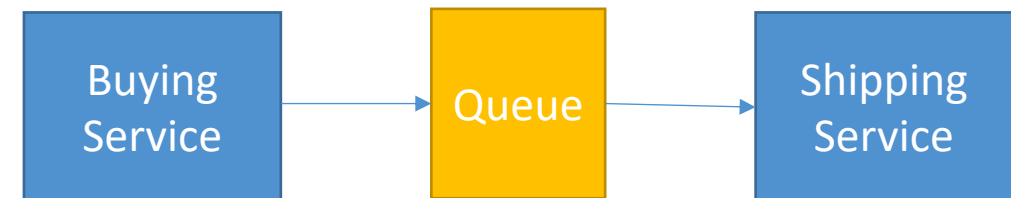
Section Introduction

- When we start deploying multiple applications, they will inevitably need to communicate with one another
- There are two patterns of application communication

1) Synchronous communications (application to application)



2) Asynchronous / Event based (application to queue to application)

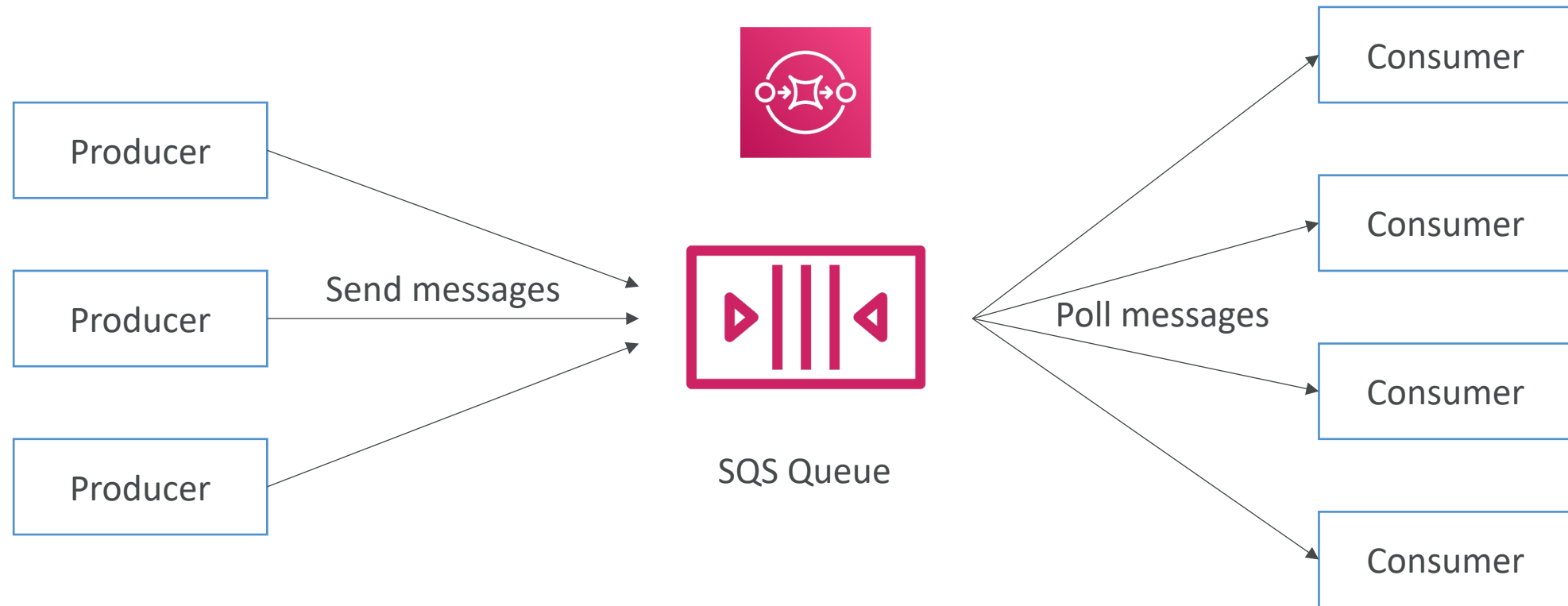


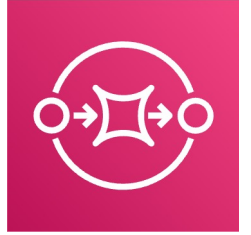
Section Introduction

- Synchronous between applications can be problematic if there are sudden spikes of traffic
- What if you need to suddenly encode 1000 videos but usually it's 10?
- In that case, it's better to **decouple** your applications,
 - using SQS: queue model
 - using SNS: pub/sub model
 - using Kinesis: real-time streaming model
- These services can scale independently from our application!

Amazon SQS

What's a queue?



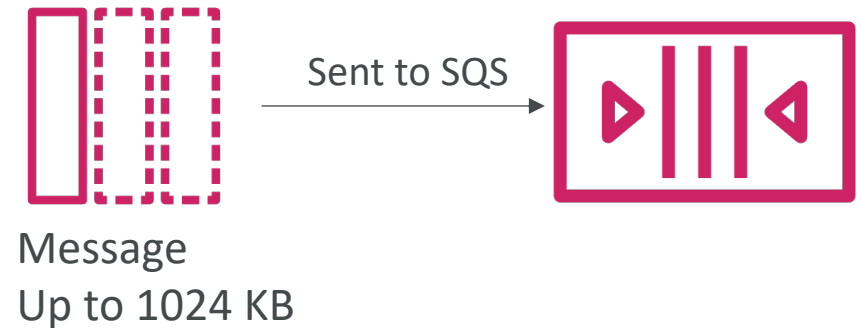


Amazon SQS – Standard Queue

- Oldest offering (over 10 years old)
- Fully managed service, used to **decouple applications**
- Attributes:
 - Unlimited throughput, unlimited number of messages in queue
 - Default retention of messages: 4 days, maximum of 14 days
 - Low latency (<10 ms on publish and receive)
 - Limitation of 1,024 KB per message sent
- Can have duplicate messages (at least once delivery, occasionally)
- Can have out of order messages (best effort ordering)

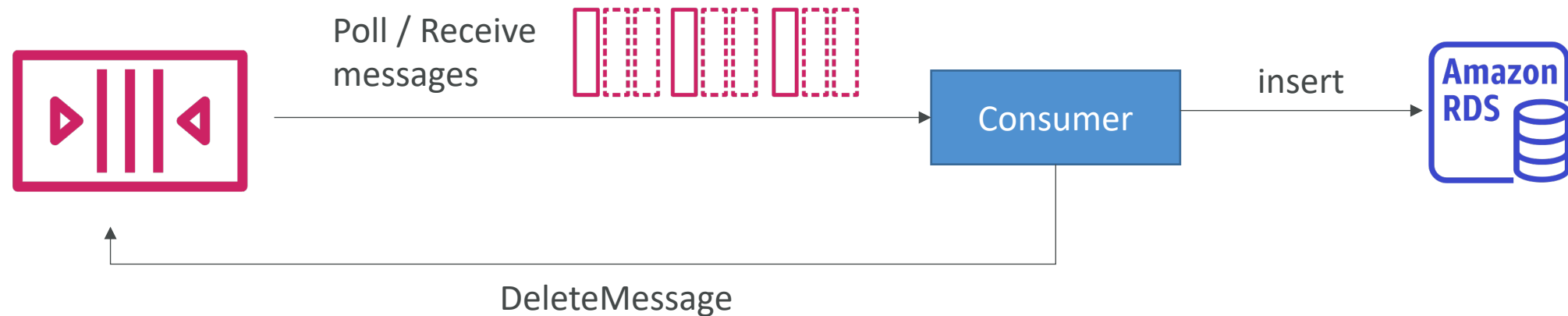
SQS – Producing Messages

- Produced to SQS using the SDK (SendMessage API)
- The message is **persisted** in SQS until a consumer deletes it
- Message retention: default 4 days, up to 14 days
- Example: send an order to be processed
 - Order id
 - Customer id
 - Any attributes you want
- SQS standard: unlimited throughput

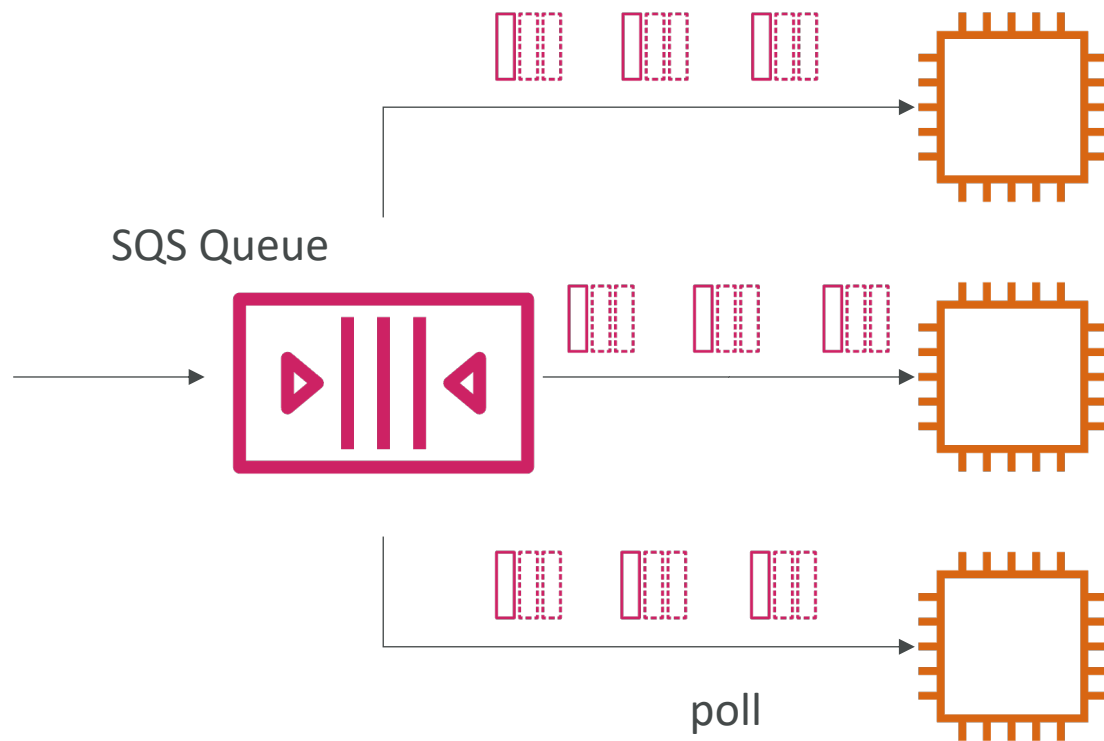


SQS – Consuming Messages

- Consumers (running on EC2 instances, servers, or AWS Lambda)...
- Poll SQS for messages (receive up to 10 messages at a time)
- Process the messages (example: insert the message into an RDS database)
- Delete the messages using the DeleteMessage API

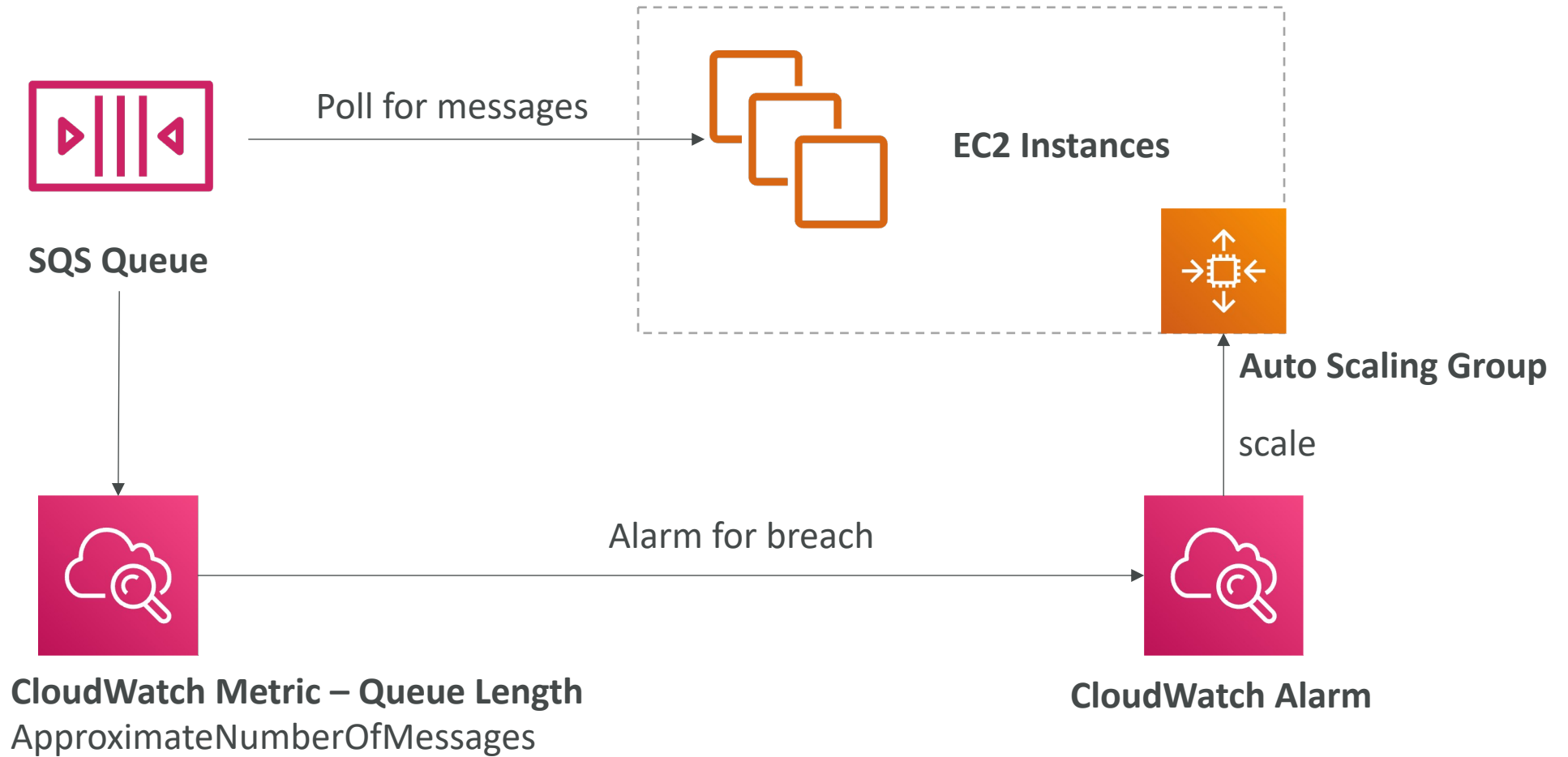


SQS – Multiple EC2 Instances Consumers

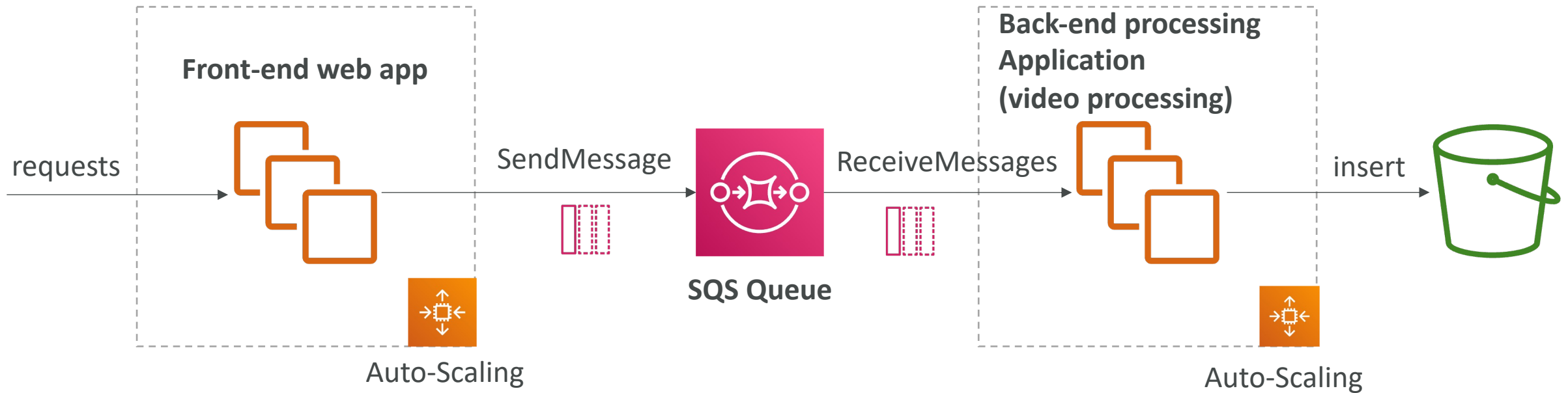


- Consumers receive and process messages in parallel
- At least once delivery
- Best-effort message ordering
- Consumers delete messages after processing them
- We can scale consumers horizontally to improve throughput of processing

SQS with Auto Scaling Group (ASG)



SQS to decouple between application tiers

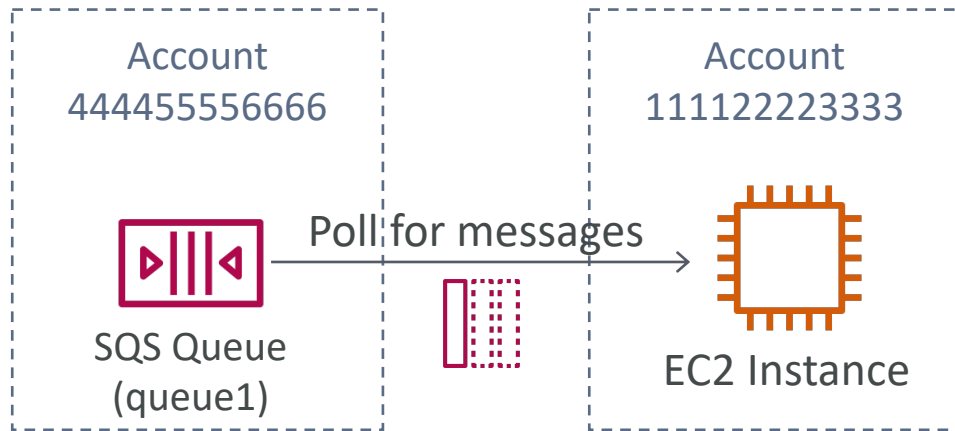


Amazon SQS - Security

- **Encryption:**
 - In-flight encryption using HTTPS API
 - At-rest encryption using KMS keys
 - Client-side encryption if the client wants to perform encryption/decryption itself
- **Access Controls:** IAM policies to regulate access to the SQS API
- **SQS Access Policies** (similar to S3 bucket policies)
 - Useful for cross-account access to SQS queues
 - Useful for allowing other services (SNS, S3...) to write to an SQS queue

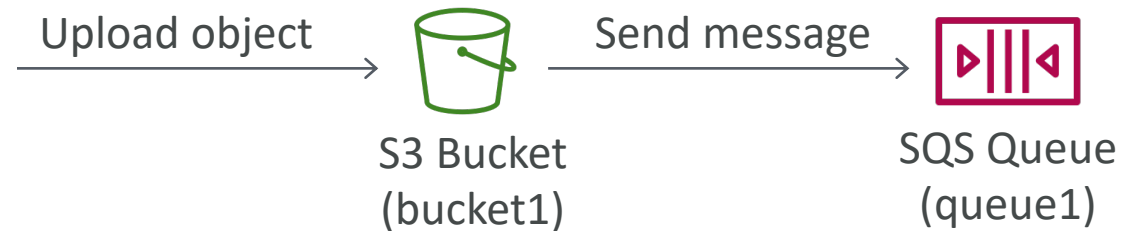
SQS Queue Access Policy

Cross Account Access



```
{
  "Version": "2012-10-17",
  "Statement": [{
    "Effect": "Allow",
    "Principal": { "AWS": [ "111122223333" ] },
    "Action": [ "sqs:ReceiveMessage" ],
    "Resource": "arn:aws:sqs:us-east-1:444455556666:queue1"
  }]
}
```

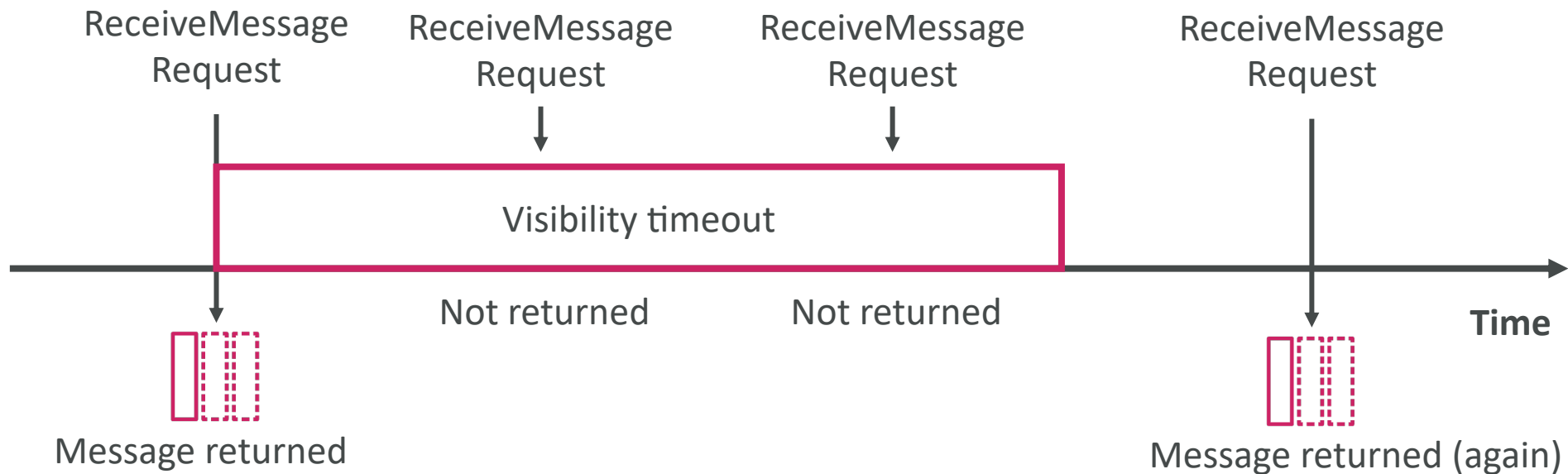
Publish S3 Event Notifications To SQS Queue



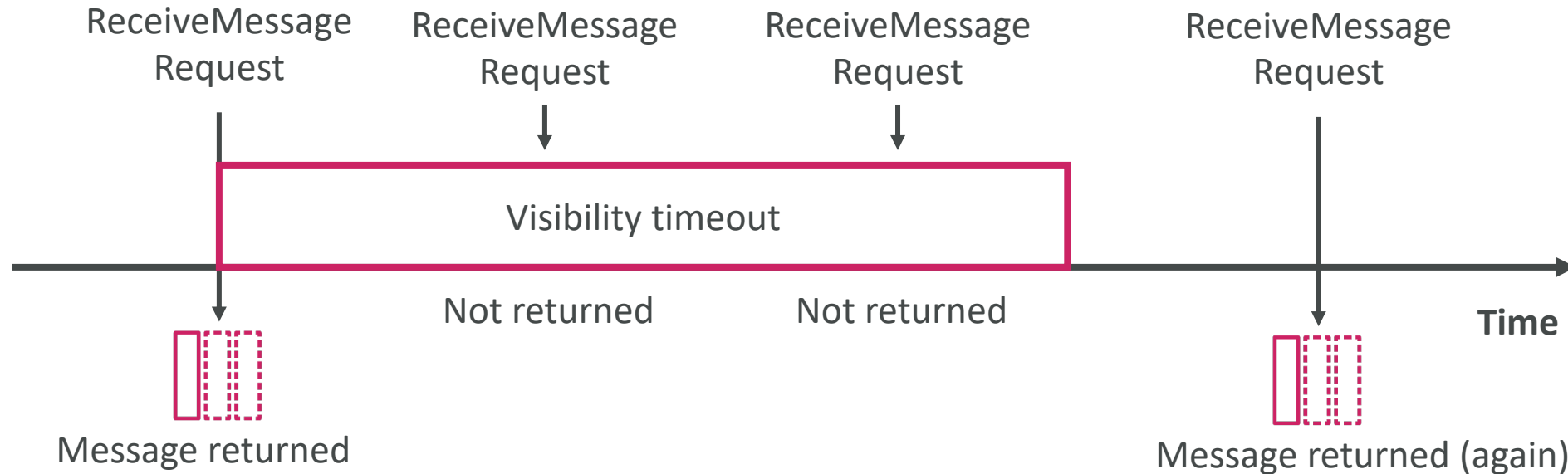
```
{
  "Version": "2012-10-17",
  "Statement": [{
    "Effect": "Allow",
    "Principal": { "AWS": "*" },
    "Action": [ "sqs:SendMessage" ],
    "Resource": "arn:aws:sqs:us-east-1:444455556666:queue1",
    "Condition": {
      "ArnLike": { "aws:SourceArn": "arn:aws:s3:*:*:bucket1" },
      "StringEquals": { "aws:SourceAccount": "<bucket1_owner_account_id>" }
    }
  }]
}
```

SQS – Message Visibility Timeout

- After a message is polled by a consumer, it becomes **invisible** to other consumers
- By default, the “message visibility timeout” is **30 seconds**
- That means the message has 30 seconds to be processed
- After the message visibility timeout is over, the message is “visible” in SQS



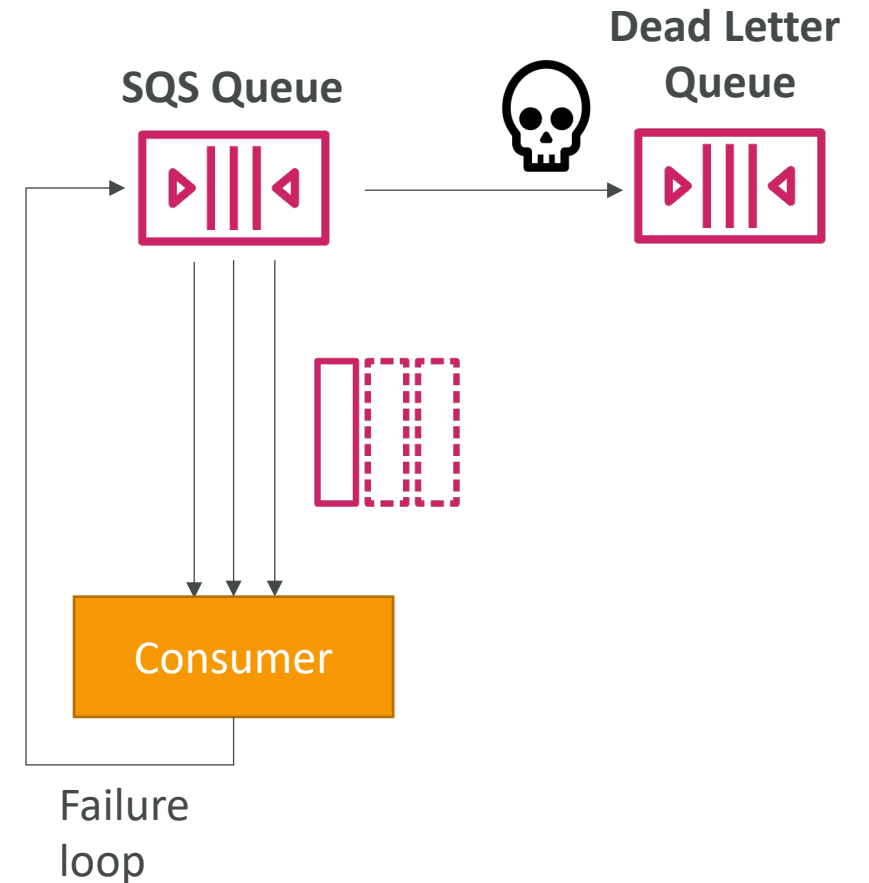
SQS – Message Visibility Timeout



- If a message is not processed within the visibility timeout, it will be processed **twice**
- A consumer could call the **ChangeMessageVisibility** API to get more time
- If visibility timeout is high (hours), and consumer crashes, re-processing will take time
- If visibility timeout is too low (seconds), we may get duplicates

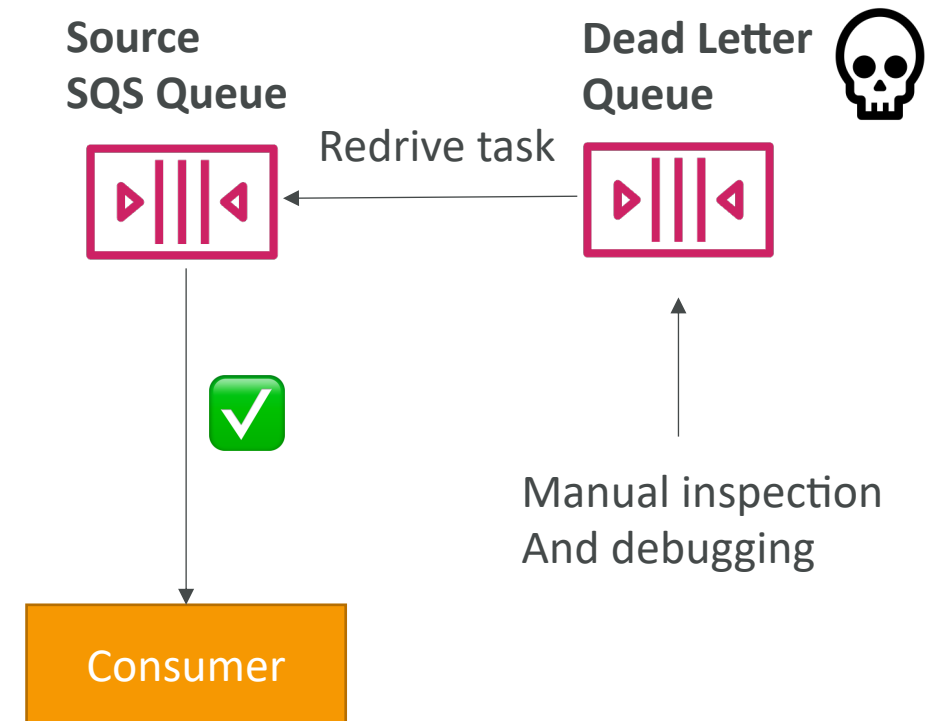
Amazon SQS – Dead Letter Queue (DLQ)

- If a consumer fails to process a message within the Visibility Timeout... the message goes back to the queue!
- We can set a threshold of how many times a message can go back to the queue
- After the **MaximumReceives** threshold is exceeded, the message goes into a Dead Letter Queue (DLQ)
- Useful for debugging!
- DLQ of a FIFO queue must also be a FIFO queue
- DLQ of a Standard queue must also be a Standard queue
- Make sure to process the messages in the DLQ before they expire:
 - Good to set a retention of 14 days in the DLQ



SQS DLQ – Redrive to Source

- Feature to help consume messages in the DLQ to understand what is wrong with them
- When our code is fixed, we can redrive the messages from the DLQ back into the source queue (or any other queue) in batches without writing custom code



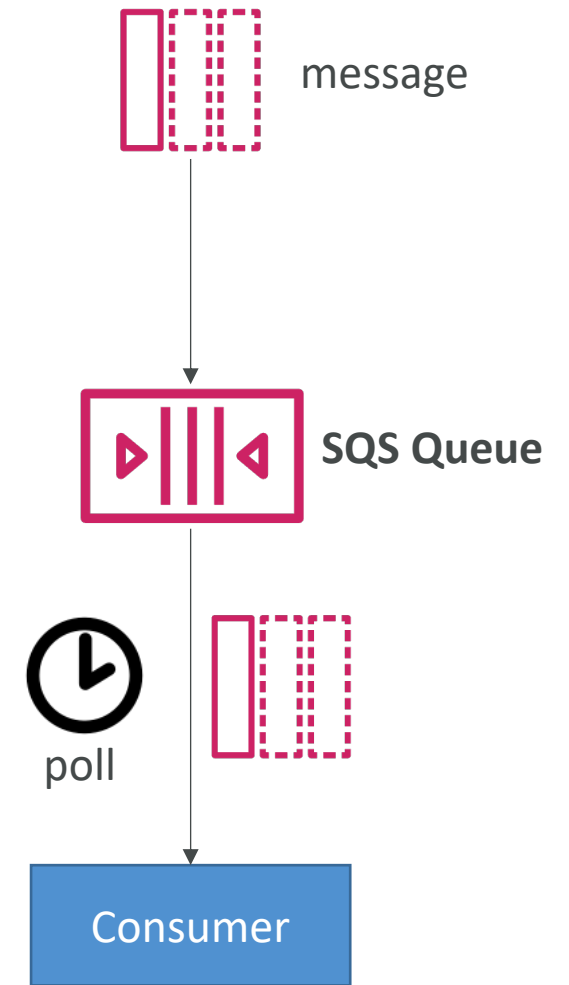
Amazon SQS – Delay Queue

- Delay a message (consumers don't see it immediately) up to 15 minutes
- Default is 0 seconds (message is available right away)
- Can set a default at queue level
- Can override the default on send using the **DelaySeconds** parameter



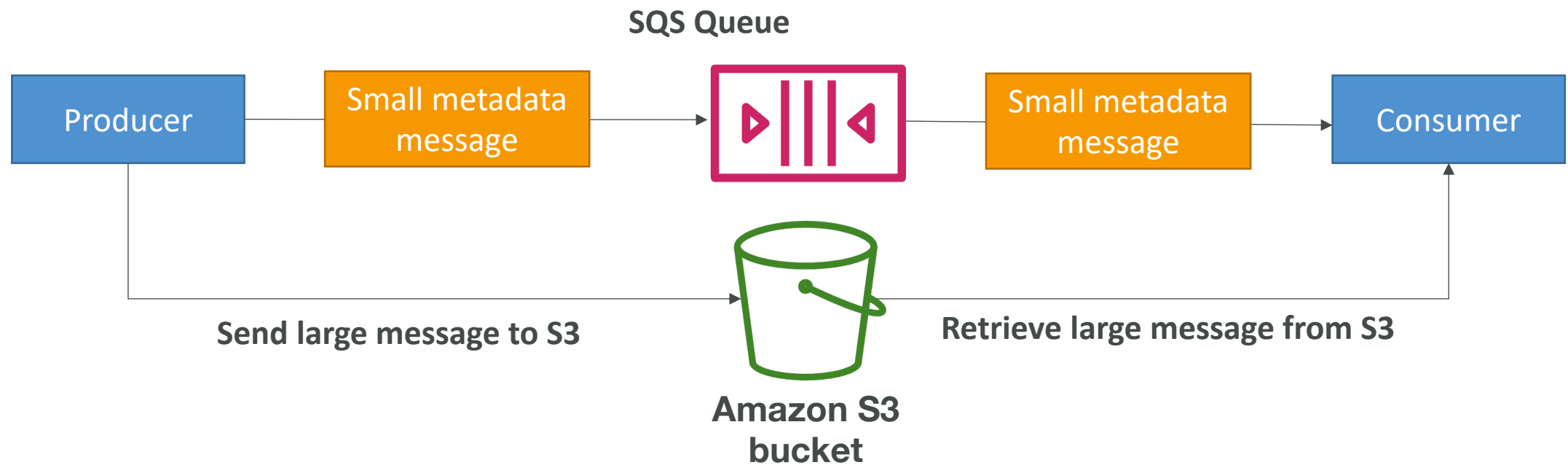
Amazon SQS - Long Polling

- When a consumer requests messages from the queue, it can optionally “wait” for messages to arrive if there are none in the queue
- This is called Long Polling
- LongPolling decreases the number of API calls made to SQS while increasing the efficiency and decreasing the latency of your application.
- The wait time can be between 1 sec to 20 sec (20 sec preferable)
- Long Polling is preferable to Short Polling
- Long polling can be enabled at the queue level or at the API level using `ReceiveMessageWaitTimeSeconds`



SQS Extended Client

- Message size limit is 1024 KB, how to send large messages, e.g. 1 GB?
- Using the SQS Extended Client (Java Library)

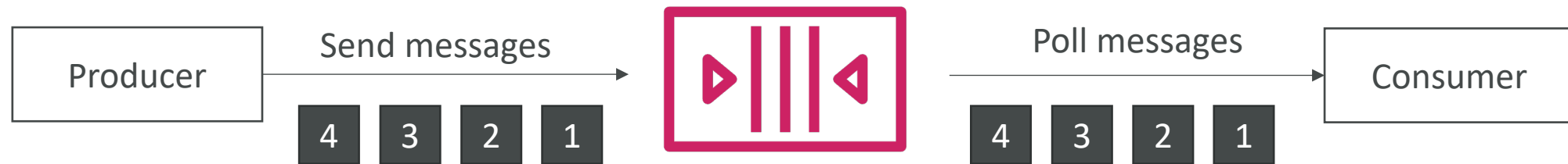


SQS – Must know API

- CreateQueue (MessageRetentionPeriod), DeleteQueue
- PurgeQueue: delete all the messages in queue
- SendMessage (DelaySeconds), ReceiveMessage, DeleteMessage
- MaxNumberOfMessages: default 1, max 10 (for ReceiveMessage API)
- ReceiveMessageWaitTimeSeconds: Long Polling
- ChangeMessageVisibility: change the message timeout
- Batch APIs for SendMessage, DeleteMessage, ChangeMessageVisibility helps decrease your costs

Amazon SQS – FIFO Queue

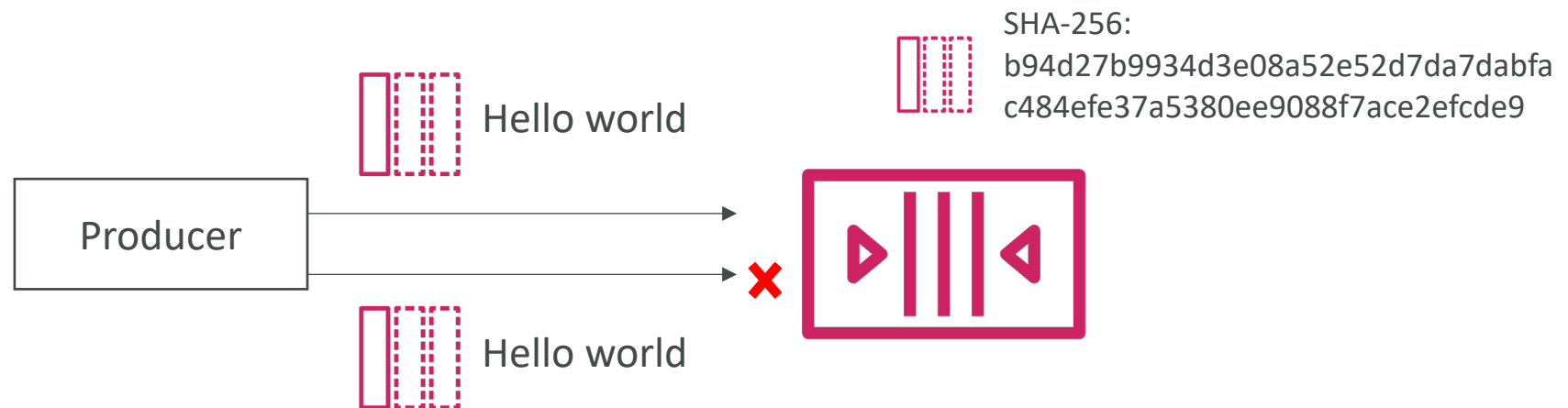
- FIFO = First In First Out (ordering of messages in the queue)



- Limited throughput: 300 msg/s without batching, 3000 msg/s with
- Exactly-once send capability (by removing duplicates using Deduplication ID)
- Messages are processed in order by the consumer
- Ordering** by Message Group ID (all messages in the same group are ordered) – mandatory parameter

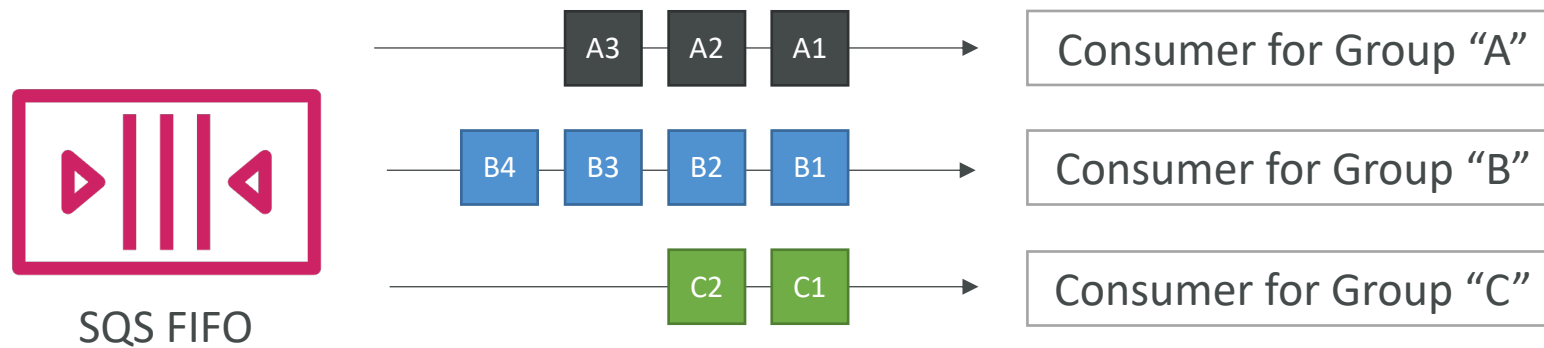
SQS FIFO – Deduplication

- De-duplication interval is 5 minutes
- Two de-duplication methods:
 - Content-based deduplication: will do a SHA-256 hash of the message body
 - Explicitly provide a Message Deduplication ID



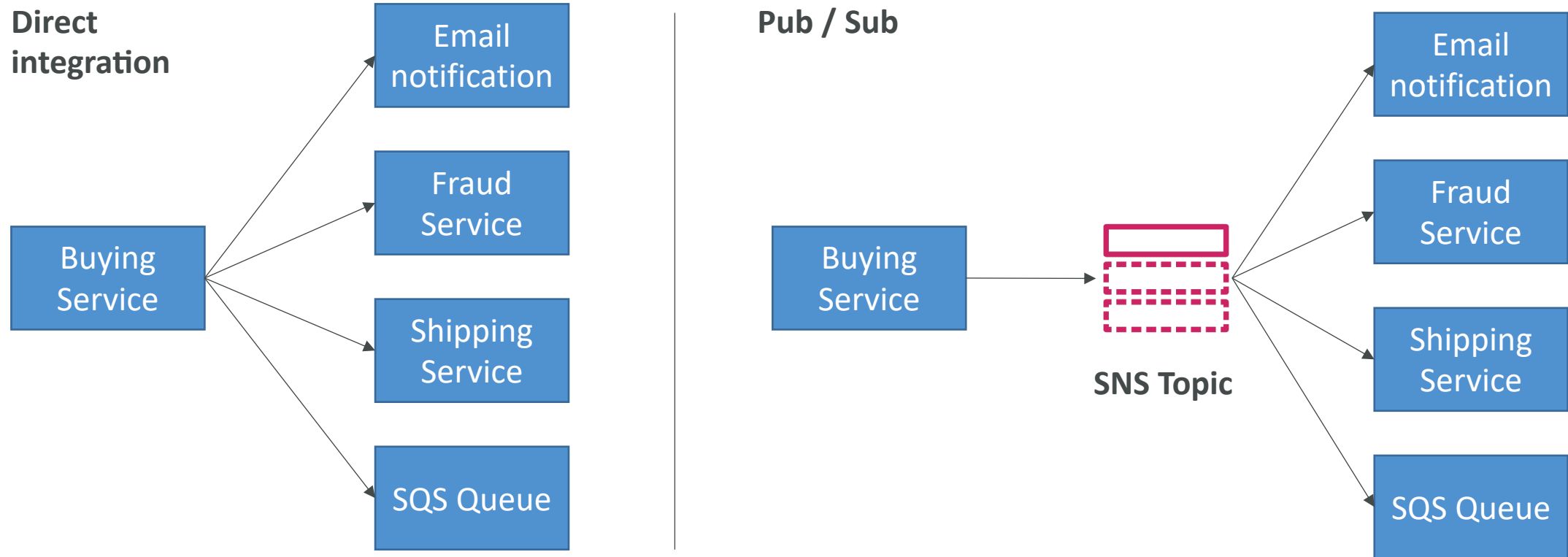
SQS FIFO – Message Grouping

- If you specify the same value of **MessageGroupId** in an SQS FIFO queue, you can only have one consumer, and all the messages are in order
- To get ordering at the level of a subset of messages, specify different values for **MessageGroupId**
 - Messages that share a common Message Group ID will be in order within the group
 - Each Group ID can have a different consumer (parallel processing!)
 - Ordering across groups is not guaranteed



Amazon SNS

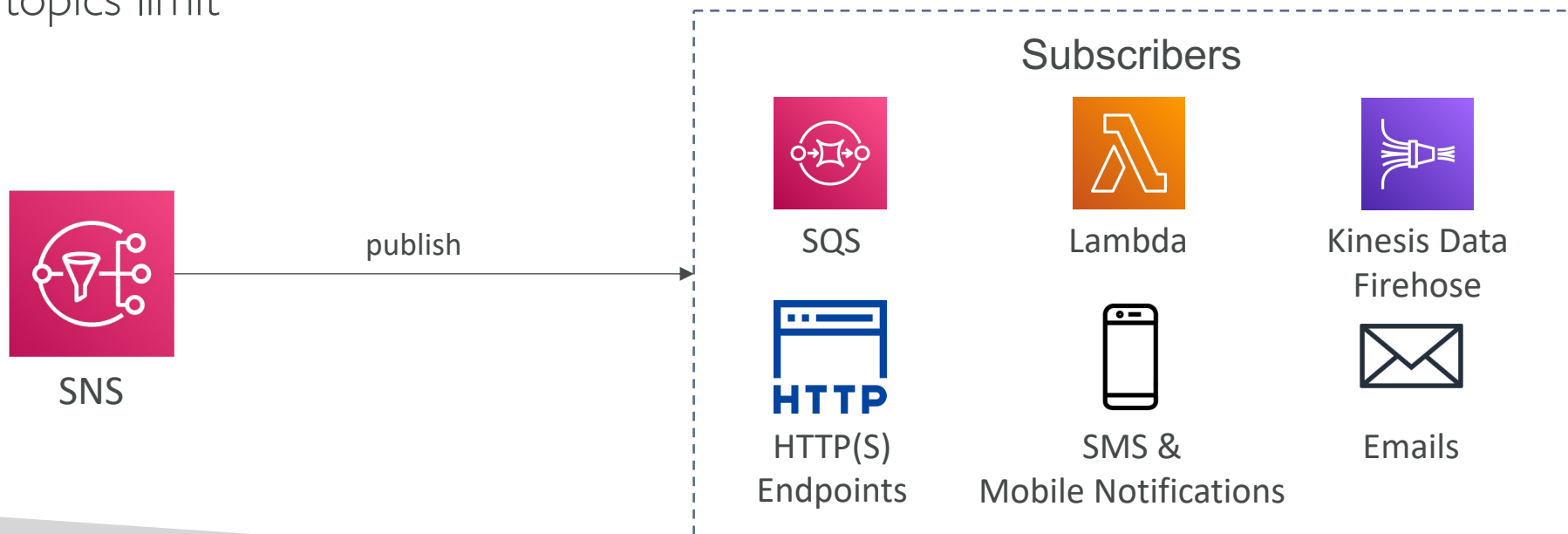
- What if you want to send one message to many receivers?



Amazon SNS

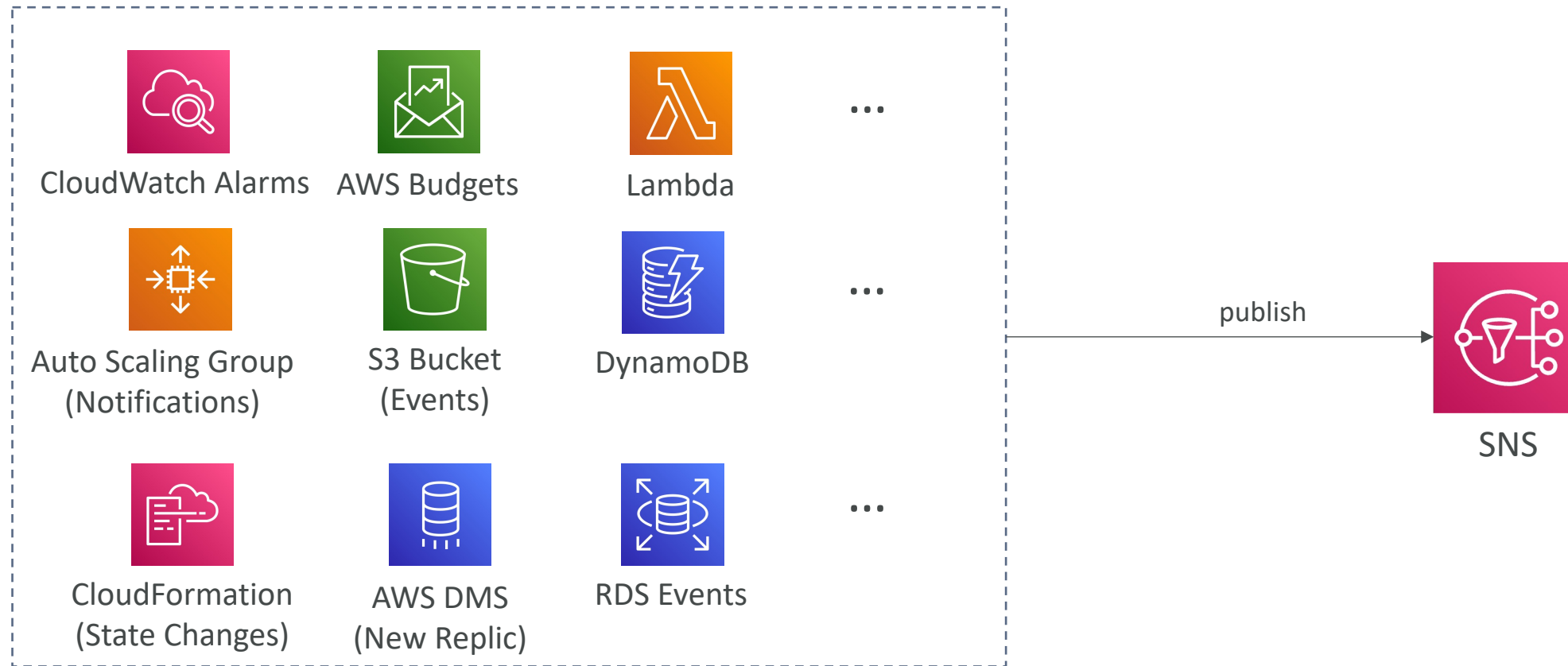


- The “event producer” only sends message to one SNS topic
- As many “event receivers” (subscriptions) as we want to listen to the SNS topic notifications
- Each subscriber to the topic will get all the messages (note: new feature to filter messages)
- Up to 12,500,000 subscriptions per topic
- 100,000 topics limit



SNS integrates with a lot of AWS services

- Many AWS services can send data directly to SNS for notifications



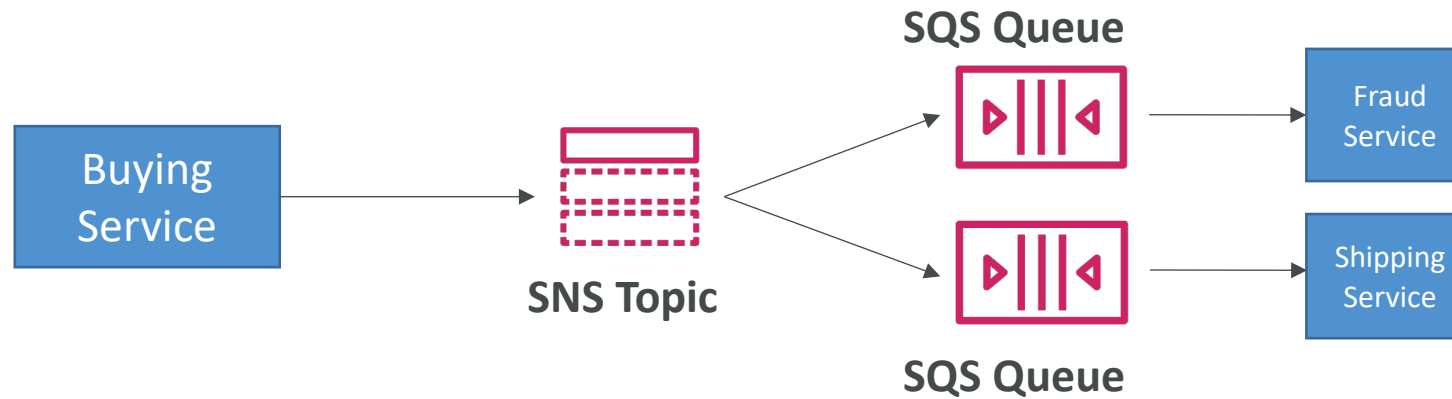
Amazon SNS – How to publish

- Topic Publish (using the SDK)
 - Create a topic
 - Create a subscription (or many)
 - Publish to the topic
- Direct Publish (for mobile apps SDK)
 - Create a platform application
 - Create a platform endpoint
 - Publish to the platform endpoint
 - Works with Google GCM, Apple APNS, Amazon ADM...

Amazon SNS – Security

- **Encryption:**
 - In-flight encryption using HTTPS API
 - At-rest encryption using KMS keys
 - Client-side encryption if the client wants to perform encryption/decryption itself
- **Access Controls:** IAM policies to regulate access to the SNS API
- **SNS Access Policies** (similar to S3 bucket policies)
 - Useful for cross-account access to SNS topics
 - Useful for allowing other services (S3...) to write to an SNS topic

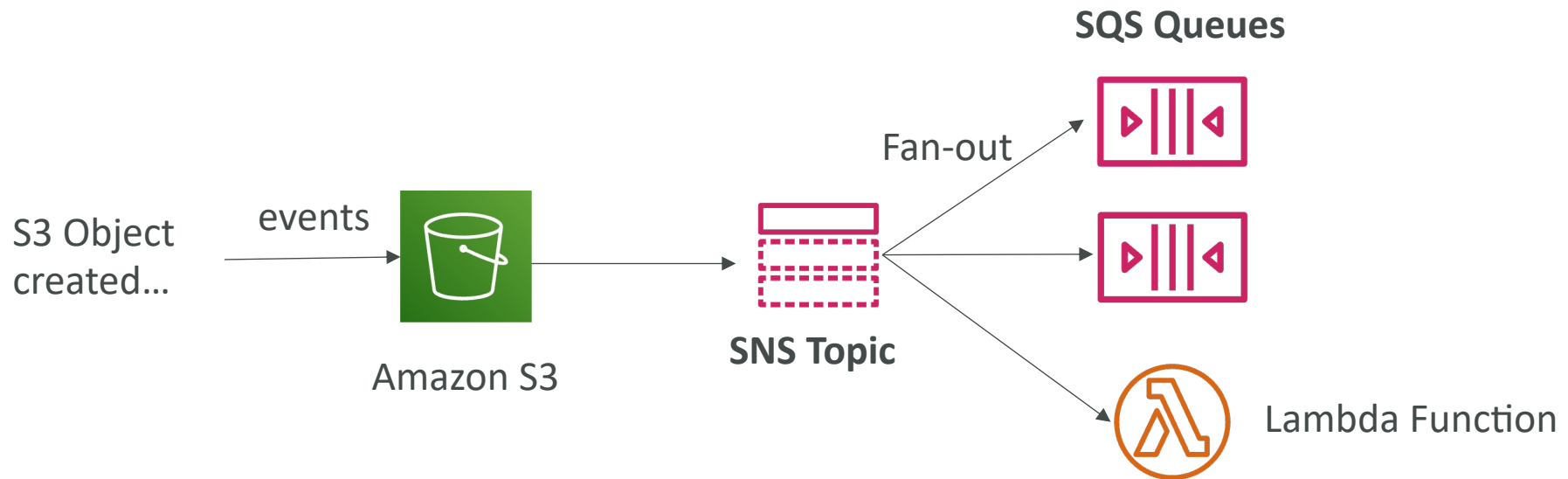
SNS + SQS: Fan Out



- Push once in SNS, receive in all SQS queues that are subscribers
- Fully decoupled, no data loss
- SQS allows for: data persistence, delayed processing and retries of work
- Ability to add more SQS subscribers over time
- Make sure your SQS queue **access policy** allows for SNS to write
- Cross-Region Delivery: works with SQS Queues in other regions

Application: S3 Events to multiple queues

- For the same combination of: **event type** (e.g. object create) and **prefix** (e.g. images/) you can only have one S3 Event rule
- If you want to send the same S3 event to many SQS queues, use fan-out



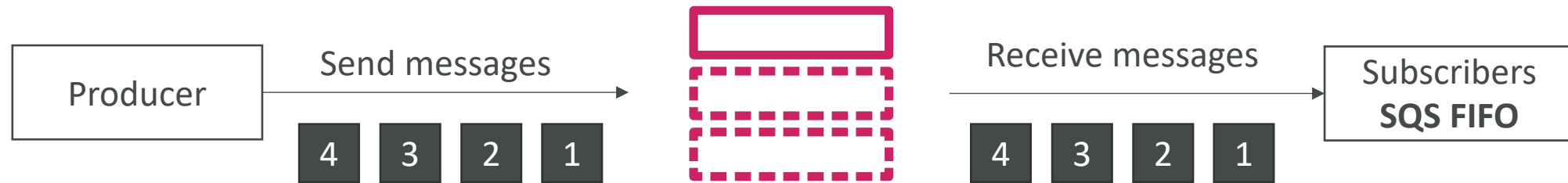
Application: SNS to Amazon S3 through Kinesis Data Firehose

- SNS can send to Kinesis and therefore we can have the following solutions architecture:



Amazon SNS – FIFO Topic

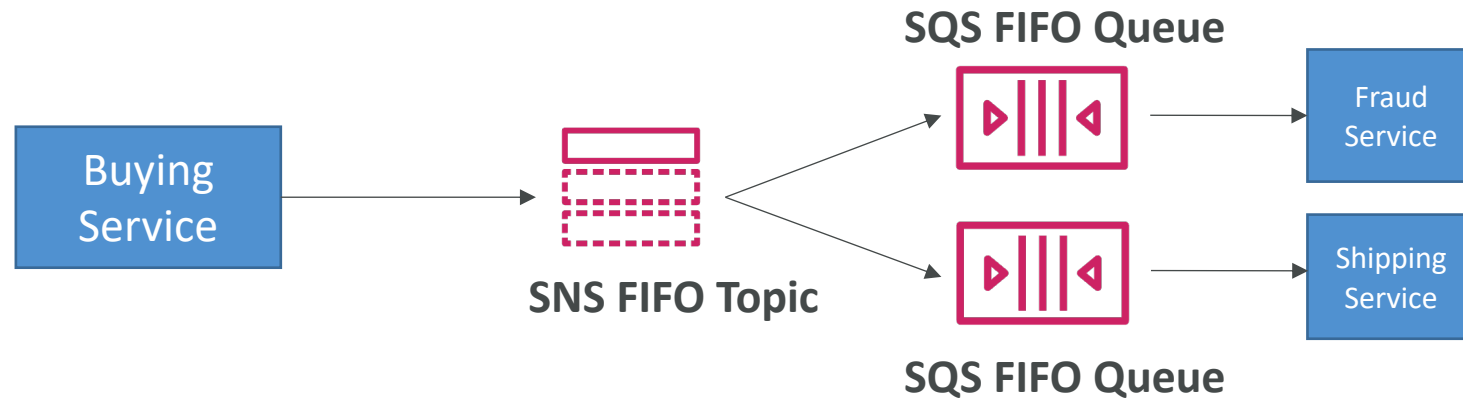
- FIFO = First In First Out (ordering of messages in the topic)



- Similar features as SQS FIFO:
 - Ordering** by Message Group ID (all messages in the same group are ordered)
 - Deduplication** using a Deduplication ID or Content Based Deduplication
- Can have SQS Standard and FIFO queues as subscribers
- Limited throughput (same throughput as SQS FIFO)

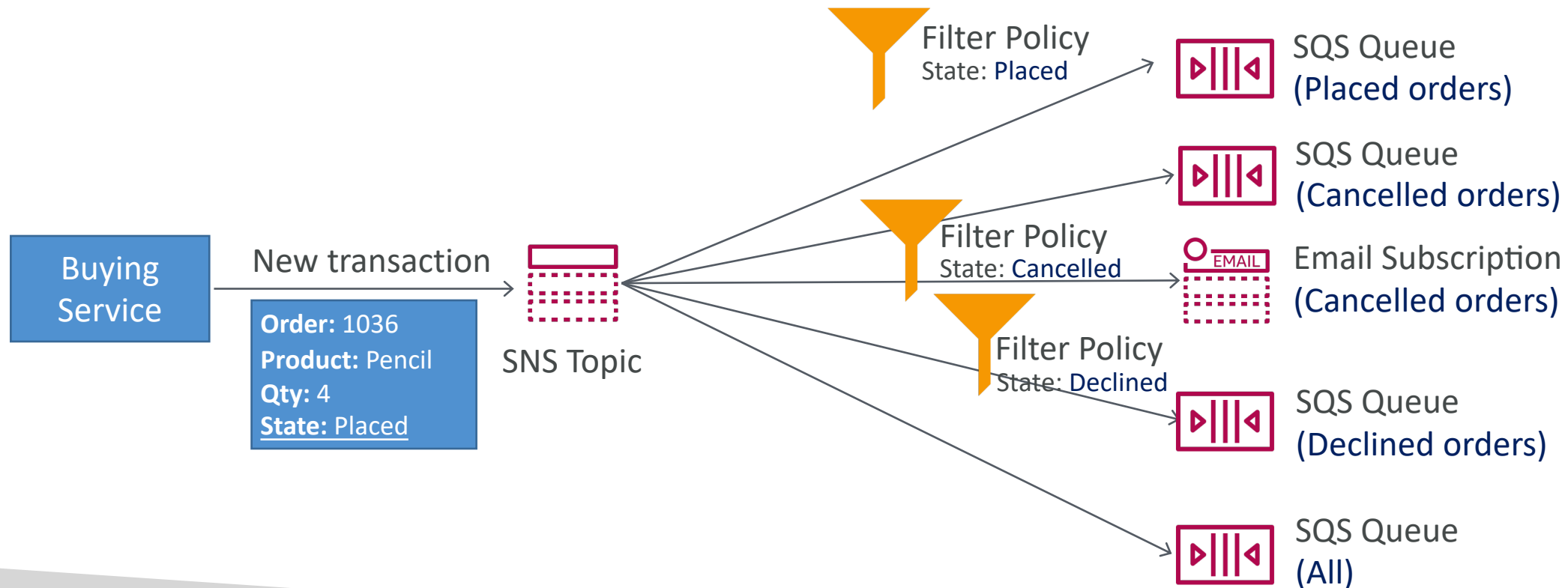
SNS FIFO + SQS FIFO: Fan Out

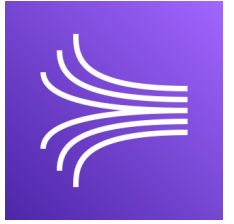
- In case you need fan out + ordering + deduplication



SNS – Message Filtering

- JSON policy used to filter messages sent to SNS topic's subscriptions
- If a subscription doesn't have a filter policy, it receives every message





Amazon Kinesis Data Streams

- Collect and store streaming data in **real-time**





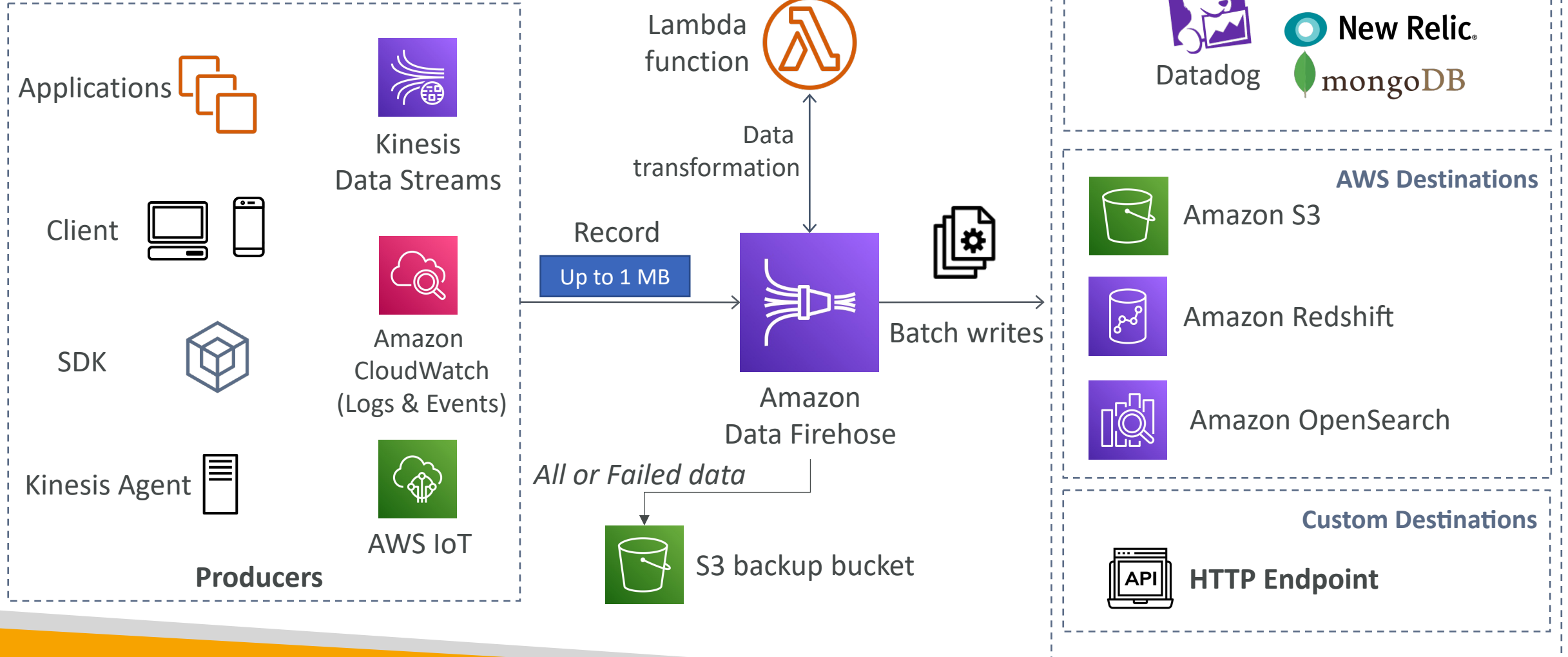
Kinesis Data Streams

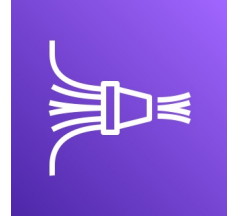
- Retention between up to 365 days
- Ability to reprocess (replay) data by consumers
- Data can't be deleted from Kinesis (until it expires)
- Data up to 10MiB (typical use case is lot of “small” **real-time data**)
- Data ordering guarantee for data with the same “Partition ID”
- At-rest KMS encryption, in-flight HTTPS encryption
- Kinesis Producer Library (KPL) to write an optimized producer application
- Kinesis Client Library (KCL) to write an optimized consumer application

Kinesis Data Streams – Capacity Modes

- **Provisioned mode:**
 - Choose number of shards
 - Each shard gets 1MB/s in (or 1000 records per second)
 - Each shard gets 2MB/s out
 - Scale manually to increase or decrease the number of shards
 - You pay per shard provisioned per hour
- **On-demand mode:**
 - No need to provision or manage the capacity
 - Default capacity provisioned (4 MB/s in or 4000 records per second)
 - Scales automatically based on observed throughput peak during the last 30 days
 - Pay per stream per hour & data in/out per GB

Amazon Data Firehose





Amazon Data Firehose

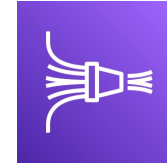
- *Note: used to be called “Kinesis Data Firehose”*
- Fully Managed Service
 - Amazon Redshift / Amazon S3 / Amazon OpenSearch Service
 - 3rd party: Splunk / MongoDB / Datadog / NewRelic / ...
 - Custom HTTP Endpoint
- Automatic scaling, serverless, pay for what you use
- **Near Real-Time** with buffering capability based on size / time
- Supports CSV, JSON, Parquet, Avro, Raw Text, Binary data
- Conversions to Parquet / ORC, compressions with gzip / snappy
- Custom data transformations using AWS Lambda (ex: CSV to JSON)

Kinesis Data Streams vs Amazon Data Firehose



Kinesis Data Streams

- Streaming data collection
- Producer & Consumer code
- Real-time
- Provisioned / On-Demand mode
- Data storage up to 365 days
- Replay Capability

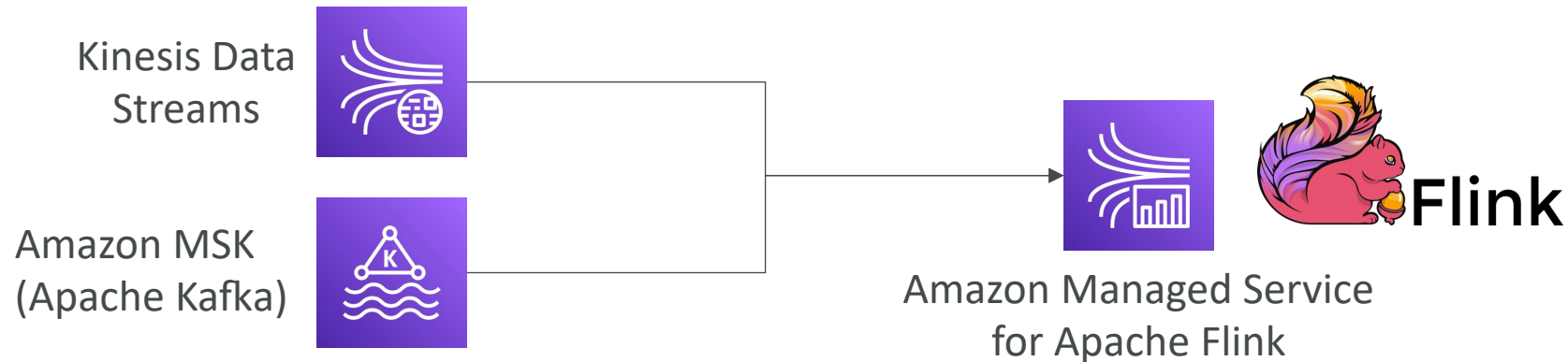


Amazon Data Firehose

- Load streaming data into S3 / Redshift / OpenSearch / 3rd party / custom HTTP
- Fully managed
- Near real-time
- Automatic scaling
- No data storage
- Doesn't support replay capability

Amazon Managed Service for Apache Flink

- *Previously named: Kinesis Data Analytics for Apache Flink*
- Flink (Java, Scala or SQL) is a framework for processing data streams



- Run any Apache Flink application on a managed cluster on AWS
 - Provisioned compute resources, parallel computation, automatic scaling
 - Application backups (implemented as checkpoints and snapshots)
 - Use any Apache Flink programming features to transform data
 - Important: Flink does not read from Amazon Data Firehose

SQS vs SNS vs Kinesis

SQS:



- Consumer “pull data”
- Data is deleted after being consumed
- Can have as many workers (consumers) as we want
- No need to provision throughput
- Ordering guarantees only on FIFO queues
- Individual message delay capability

SNS:



- Push data to many subscribers
- Up to 12,500,000 subscribers
- Data is not persisted (lost if not delivered)
- Pub/Sub
- Up to 100,000 topics
- No need to provision throughput
- Integrates with SQS for fan-out architecture pattern
- FIFO capability for SQS FIFO

Kinesis:



- Standard: pull data
 - 2 MB per shard
- Enhanced-fan out: push data
 - 2 MB per shard per consumer
- Possibility to replay data
- Meant for real-time big data, analytics and ETL
- Ordering at the shard level
- Data expires after X days
- Provisioned mode or on-demand capacity mode